IT Handover — منصة التحول للذكاء الاصطناعي المساعد

Everything the IT team needs to stand up the production deployment: database, roles, authentication hooks, and operational notes. The repository is a Next.js 14 (App Router, TypeScript) application with a PostgreSQL schema managed by Prisma.

---

1. Runtime & prerequisites

Component	Requirement
Node.js	20 LTS or newer
PostgreSQL	**14+ recommended** (12 minimum — migration `0006` uses `ALTER TYPE … ADD VALUE` inside a transaction)
Package manager	npm (lockfile committed)

```
cp .env.example .env          # then fill in the (REQUIRED) values — see §2
npm ci
npm run db:setup              # migrate deploy (0001 → 0011) + generate + seed
npm run build && npm start    # serves on PORT (default 3000)
```

Or, fully containerised (Postgres + app, migrations + seed on first boot):

```
cp .env.example .env          # fill in the (REQUIRED) values
docker compose up --build     # app on :3000, Postgres on :5432
```

The seed loads **reference data and starter accounts only** — streams, entities, phases, stages, roles/permissions, and the placeholder users. The portfolio starts **empty**; set `SEED_DEMO_ITEMS=1` only on a demo/staging database if sample items are wanted.

What IT changes: only the values in `.env` (DB URL, `SESSION_SECRET`, UAE PASS client id/secret/redirect, optional AI endpoint). No code edits are required. `NEXT_PUBLIC_*` values are build-time — set them before the build.

2. Environment variables

The single source of truth is **`.env.example`** — every variable is listed there with `(REQUIRED)` / `(build-time)` markers and inline guidance. Copy it to `.env` and fill in the blanks. Summary:

Variable	Purpose
`DATABASE_URL`	Postgres connection string (REQUIRED)
`NEXT_PUBLIC_DATA_MODE`	`api` in production (build-time); `local` = static demo
`SESSION_SECRET`	Signs the stateless session cookie — `openssl rand -hex 32` (REQUIRED)
`SESSION_TTL_HOURS`	Session lifetime (default 12; `SESSION_TTL_SECONDS` overrides)
`AUTH_PROVIDER` / `NEXT_PUBLIC_AUTH_PROVIDER`	`mock` (dev only) | `uaepass` | `workspaceone`
`BOOTSTRAP_ADMIN_EMAILS`	Auto-provision these emails as `system_admin` on first login
`OIDC_ISSUER` / `_CLIENT_ID` / `_CLIENT_SECRET` / `_REDIRECT_URI`	Workspace ONE OIDC (REQUIRED for `workspaceone`)
`STATE_API_TOKEN`	Bearer guard for `/api/state` on shared deployments
`NEXT_PUBLIC_UAEPASS_MODE`	`live` in production; `mock` for the demo (build-time)
`UAEPASS_CLIENT_ID` / `_SECRET` / `_REDIRECT_URI`	UAE PASS OIDC — see §5 (REQUIRED for live)
`AI_API_BASE_URL` / `_KEY` / `_MODEL`	Internal AI reviewer endpoint (optional)
`NEXT_PUBLIC_BASE_PATH`	Only for sub-path hosting (build-time)
`NEXT_PUBLIC_DEMO_MODE` / `_DATA`	**Keep `0` in production** — role switcher / mock data

Secrets belong in the host's secret manager — never in the repository. `NEXT_PUBLIC_*` variables are inlined at build time, so set them before `npm run build` / the Docker image build.

3. Database structure (Prisma → Postgres)

Schema: `prisma/schema.prisma` (tables/columns are snake_case via `@map`; the client uses camelCase). Migrations: `prisma/migrations/0001…0011`.

Reference data - `streams` — the three transformation streams (ids: `ops`, `strategy`, `services` — العمليات والدعم المؤسسي، العمل الحكومي الاستراتيجي، الخدمات الحكومية) with the official stream heads. - `entities` — federal entities (seeded from the reference list plus every entity present in the services catalog). - `program_phases` — program phases with editable deadlines (committee). - `exec_batches` — the launch milestones (التقييم والتهيئة، دفعات الإطلاق الأولى…السادسة، التوسع في التطبيق) with their date windows. Item start/end dates are constrained to their دفعة window. - `service_catalog` — دليل الخدمات الاتحادية (migration `0010`): `(entity_id FK, main_service, sub_service)` with a unique triple. Feeds the services-stream الخدمة/الخدمة الفرعية dropdowns, **scoped to the coordinator's entity**. Seeded from `lib/svcCatalog.json` (47 entities / ~1,855 rows); refresh by replacing the JSON and re-running the seed. - `settings` — key/value.

Team records (per entity) - `stream_owners` — per-stream owner inside the entity (unique per entity+stream). - `entity_reps` — legacy (the ممثل الجهة role is not part of the current confirmed flow; kept for compatibility).

Portfolio - `items` — every خدمة / مهمة / عملية (types `operation` and `service`; `project` / `initiative` remain in the enum for legacy data only). Carries the header fields (ops: التصنيف/نوع عملية الدعم المؤسسي؛ strategy: المحور؛ services: الخدمة), the **per-نشاط details in `activities` (JSONB, migration `0011`)** — each نشاط/خدمة فرعية with its own sector/dept/section, automation, matrix and أولوية التحول, mirrored onto the legacy flat columns (first entry) for compatibility — the دفعة assignment (`exec_batch` — a دفعة name or «للتحديد بعد الدراسة»), the returned-with-notes state (`ret_*`), and the batch-move marker (`stage_move_*` — drives the رئيس المسار notification). See `docs/CHANGES-2026-08-11.md` for the full model and rules. `wf` is the workflow enum; the current flow uses `draft → ent1 → exec` and the UI presents exactly four statuses: مسودة / قيد الاعتماد / للتعديل (returned) / معتمد. Approved entries are locked from editing. - `exec_checklist_items`, `sub_milestones` — per-item execution details. - `launch_plans`, `item_launch_plans`, `launches`, `item_launches` — **legacy** (the managed launch-plan concept was removed from the flow; دفعات الإطلاق are the `exec_batches` reference rows).

Governance - `log_entries` — the approval/action audit trail shown in السجل. - `nominations`, `fundings` / `funding_cancellations` — **legacy** (the nomination/funding concepts were removed from the confirmed flow).

Access control (RBAC), audit & notifications (migrations `0007` – `0009`) - `roles` — backend access roles keyed by a stable `code` (nine codes, see §4). `users.role` keeps the *legacy UI key* (`admin`, `coord`, `entity`, `path`, `ai`) as a plain string for the client screens; server enforcement goes through `user_roles` / `role_permissions`. - `permissions` — stable permission codes (`items:approve`, `funding:cancel`, …). - `role_permissions` — role → permission matrix (seeded, editable). - `user_roles` — role assignment per user (the app enforces one role per user). - `user_entity_scopes` / `user_stream_scopes` — per-user data scopes; the API filters every item query by them (`lib/security/rbac.ts`). A coordinator with several `user_stream_scopes` rows gets a stream switcher in the header (`/api/auth/me` returns `streamScopes`). - `audit_logs` — server-side audit trail written inside the same transaction as every enforced mutation (actor, action, resource, entity/stream, IP, user-agent, metadata). - `role_assignment_rules` — pre-configured email→role(+scopes) mappings; applied automatically on the user's first login (team setup and `/api/admin/role-rules` create them). - `sessions` — **removed** (migration `0009`). Sessions are now stateless HMAC-signed httpOnly cookies (`lib/security/session.ts`), signed with `SESSION_SECRET`; nothing is stored server-side. - `notifications` — persisted الإشعارات, targeted to a user or broadcast to a role / entity / stream, with `kind`, title/body, and `read_at`. The client currently derives notifications from data state; this table lets IT persist and push them (email/SMS) from the API layer.

Demo sync - `app_state` — single JSON blob used by `/api/state` for the demo persistence. Harmless in production; can stay empty.

The full table list (with columns) lives in `prisma/schema.prisma`; the versioned DDL is in `prisma/migrations/0001…0011`.

4. Role management

Access is enforced server-side: nine backend role codes (in `roles`) carry the permission matrix; the client keeps rendering its four UI roles + the admin console via the legacy `users.role` key. Mapping:

Backend code	Arabic	UI role	Scope enforced by the API
`system_admin`	مشرف النظام	`admin`	global (bypasses permission checks)
`program_admin`	مدير البرنامج	`ai`	global
`ai_committee`	اللجنة الوطنية (الرئيس + الأمانة العامة)	`ai`	global, approved items only
`stream_owner`	رئيس المسار / نائب رئيس المسار	`path` / `deputy`	own stream across all entities — **the ent1 approver**
`entity_coordinator`	منسق المسار في الجهة	`coord`	own entity + own stream(s), incl. drafts
`entity_admin`	مسؤول الجهة	`entity`	own entity + entity updates (legacy)
`entity_representative`	ممثل الجهة	`entity`	**legacy — not in the current flow**
`viewer`	مستعرض	—	read-only within scopes
`auditor`	مدقق	—	read-only + `audit:view`

Role → permission assignments live in `role_permissions` (seeded from `prisma/seed.ts`). **Approval sits with the stream:** `stream_owner` (رئيس المسار ونائبه) carries `items:approve` / `items:reject` and is the sole approver at the `ent1` gate — matching the client, where اعتماد / إعادة للتعديل / طلب تفاصيل actions render only for the stream head and deputy. `entity_representative` is view-level legacy and holds no approval permissions.

Who provisions whom: `system_admin` manages **all accounts for all entities** and role rules (`/api/admin/users`, `/api/admin/role-rules`) — stream heads and deputies (`stream_owner` + stream scope), coordinators (`entity_coordinator` + entity/stream scopes), and the national committee (`ai_committee`). There is no self-service team-setup screen. `BOOTSTRAP_ADMIN_EMAILS` auto-provisions the very first system admin(s) on login.

Seeded starter accounts (in `users`, keyed by email on the `@aigp.gov.ae` placeholder domain): the system admin (`admin@…`), the national committee, the three stream heads (`head.<stream>@…`), and one coordinator per stream (`coord.<stream>@…`). Each is seeded active with its backend role in `user_roles` plus the matching entity/stream scopes. To go live, re-point each account's `email` to the verified UAE PASS identity (or deactivate and create real ones).

Rules the application assumes (enforce when provisioning users): - `coord` **must** have `entity_id`; `coord`, `path` and `deputy` **must** have `stream_id`; `ai` has neither. - The `path` / `deputy` users per stream are the official stream head and deputy (head names pre-seeded on `streams.head_name`); attach their emails when known. - Deactivate (never delete) users via `is_active = false` so the audit log keeps valid author names.

Data visibility implemented by the app (for reference): - Drafts (`wf = 'draft'`) are visible **only** to the coordinator. - `ent1` (قيد الاعتماد) is visible to the coordinator and the stream head/deputy, who approve or return it. - The committee sees approved entries only (read-only, national view).

5. Server API (enforced endpoints)

Every route below runs `requireAuthUser` → `assertPermission` → `assertItemAccess` (entity/stream scope), mutates inside a transaction and writes an `audit_logs` row (`lib/security/`):

- **Auth**: `GET /api/auth/login` (provider dispatch: mock / uaepass / workspaceone), `POST /api/auth/logout` , `GET /api/auth/me` (roles + permissions + scopes), `GET /callback` (Workspace ONE OIDC redirect), `GET /api/auth/uaepass/login|callback` (UAE PASS, §6).
- **Items**: `GET|POST /api/items` , `GET|PATCH|DELETE /api/items/:id` , `POST /api/items/:id/submit|approve|reject|return` (workflow actions with log entries + audit).
- **Services catalog**: `GET /api/svc-catalog` — دليل الخدمات الاتحادية for the services-stream dropdowns, **scoped server-side to the session user's entity**; only global roles may pass `?entityId=` . Requires `items:view` .
- **Portfolio reads (legacy)**: `GET /api/funding` , `GET /api/nominations` , `GET /api/launch-plans` — scope-filtered; kept for compatibility only (the concepts are out of the current flow).
- **Team records (legacy)**: `POST /api/team/register` — kept for compatibility; provisioning is done by the admin via `/api/admin/*` .
- **Admin**: `GET|POST /api/admin/users` , `GET|PATCH /api/admin/users/:id` , `POST /api/admin/users/:id/enable|disable` , `POST /api/admin/users/:id/roles` (+ `DELETE …/roles/:roleId`), `GET /api/admin/roles|permissions|entities|streams` , `GET|POST|DELETE /api/admin/role-rules` , `GET /api/admin/audit-logs` .
- **Ops**: `GET /api/health` (liveness), `GET /api/ready` (DB probe).
- **State blob (enforced)**: `GET|PUT /api/state` — requires a signed session AND a global-scope role (`canAccessAllEntities`); scoped users get `{data:null, scoped:true}` . `PUT` additionally honors `STATE_API_TOKEN` when set. Because mock login is disabled with `NODE_ENV=production` , shared server state in production requires a real `AUTH_PROVIDER` (uaepass/workspaceone); until then each browser falls back to local storage.
- **AI reviewer (enforced)**: `POST /api/ai-review` — requires session + `ai_review:run` permission, honors `AI_REVIEW_ENABLED=false` as a kill-switch, and writes an audit log. Clients without access fall back to the built-in heuristic review.

6. Authentication (UAE PASS)

`app/api/auth/uaepass/login` and `…/callback` are fully wired: the callback validates state, exchanges the code, maps the verified identity to `users` via `ensureUserFromIdentity` (bootstrap admins + role-assignment rules, pending/disabled users are turned away), and issues the signed session cookie. The demo build bypasses this flow behind the "Sign in with UAE PASS" button. To go live: 1. Register the redirect URI with UAE PASS and set the client id/secret in the environment (`UAEPASS_*` variables), plus `AUTH_PROVIDER=uaepass` and `NEXT_PUBLIC_AUTH_PROVIDER=uaepass` (build arg). 2. Nothing else — user mapping and access gating are

implemented in `lib/security/user-access.ts`. 3. Sessions are stateless cookies (HMAC-signed with `SESSION_SECRET`, `maxAge = SESSION_TTL_HOURS`, httpOnly) via `lib/security/session.ts`; there is no sessions table. `/api/auth/me` resolves `{role, entityId, streamId, name}` plus backend roles/permissions — the client reads the role from it — the role-switcher tabs only exist when `NEXT_PUBLIC_DEMO_MODE=1`.

Invitation emails for entity reps and coordinators are drafted in `docs/email-templates.md` — wire them to the mail gateway when accounts are provisioned.

7. Operational notes

- **Notifications** are currently derived in the client from data state (submissions, approvals, returns, batch moves). The mandatory automatic email notifications (submission → stream head/deputy; approval/return → coordinator; batch move → stream head) must be wired by IT via `POST /api/notify` against the government SMTP gateway — the event → recipient matrix is in `HANDOVER.md`, templates in `docs/email-templates.md`.
- **Exports** (Excel/PowerPoint) are generated client-side; no server dependency. The Excel report fills the official workplan template (`public/assets/workplan_template.xlsx`) — the same sections entities use for bulk upload (team sheet omitted) — and the PowerPoint deck carries a branded title slide, a KPI summary, and one card-style slide per entry.
- **Backups**: standard Postgres dumps; all state is in the tables above (nothing critical lives in `app_state`).
- **Security**: run behind the government gateway/WAF; the app sets no cookies of its own in demo mode; rotate any tokens used during the handover (including the temporary GitHub deploy token used for the demo site).